Lorenz Controller Project — Context Summary

TACS Lab, UC Santa Cruz
Pulkit Dubey / Ashesh Chattopadhyay

Split: Patrick — gradient-based control (BPTT/adjoint) · You — reinforcement learning

1 Research question

Can reinforcement learning retain usable control signal past the **Lyapunov time**, the horizon where adjoint/BPTT gradients become numerically useless in a chaotic system? Not a straightforward bake-off — the comparison is specifically about what happens in the horizon regime where gradient-based optimization breaks down.

2 The setup

State $x(t) \in \mathbb{R}^3$ (Lorenz), dynamics $\dot{x} = f(x) + u(t)$, where f is the Lorenz vector field and u is an added forcing/control term. Goal: find u minimizing a cost

$$J[u] = \int_0^T \ell(x(t), u(t))\, dt$$

(e.g., stay on one wing, track a reference, suppress lobe-switching).

Lorenz system:

$$\dot{x} = \sigma(y - x), \qquad \dot{y} = x(\rho - z) - y, \qquad \dot{z} = xy - \beta z$$

Standard parameters: $\sigma = 10$, $\rho = 28$, $\beta = 8/3$. Reference Lyapunov exponent $\lambda \approx 0.9056 \Rightarrow$ Lyapunov time $\tau_\lambda \approx 1.1$ time units.

3 Why gradient-based methods fail past τ_λ

The adjoint variable $\lambda(t)$ (sensitivity) solves backward in time:

$$\dot{\lambda} = -\left(\frac{\partial f}{\partial x}\right)^\top \lambda - \frac{\partial \ell}{\partial x}, \qquad \lambda(T) = 0$$

This is the transpose of the tangent-linear equation, so $\|\lambda(t)\|$ grows like $e^{\lambda_1 (T-t)}$ integrating backward — the same mechanism as forward trajectory divergence. BPTT and the adjoint method are the same operation (chain rule through the rollout / backprop through time, invented by fluid dynamicists in the 1970s).

Two distinct failure modes, and the second is the real one:

1. **Magnitude blowup** — trivially fixable by normalizing. Not the real problem.
2. **Loss of meaning** — past a few τ_λ, J as a function of u becomes a fractal-like landscape: the derivative is locally correct but describes a "valley" one part in $e^{\lambda_1 T}$ wide. Any realistic step size overshoots it entirely. This is structurally identical to the exploding-gradient problem in RNNs unrolled over many steps.

Key reframe: the trajectory is unpredictable past τ_λ, but the *statistics* (ergodic/invariant-measure quantities — time spent on each wing, average height, switching frequency) are not. That information exists, it's just not present in the pointwise adjoint gradient $\partial x(T)/\partial u(0)$ — it's in long-horizon time averages. This is why conventional adjoints fail to compute sensitivities of ergodic averages in chaotic systems, motivating **least-squares shadowing** (Wang, Blonigan) as a third method worth including.

4 Why RL sidesteps this

Score-function/REINFORCE estimator:

$$\nabla_\theta J = \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a \mid s) \cdot R\right]$$

∇_θ never touches f — you differentiate the policy's log-density, not the dynamics. The chain-of-Jacobians product never appears, so nothing explodes. The cost instead becomes **variance**: past τ_λ, the correlation between an early action and late return decays, adding noise to the estimator — but variance is fixable with more samples/compute, unlike an exploded gradient.

	BPTT/adjoint	RL (score function)
Failure mode	Bias + ill-conditioning	Variance
Scaling in T	$\sim e^{\lambda_1 T}$	$\sim \mathrm{Var}(R)$, often polynomial
Fixable by	Nothing cheap	More samples, baselines, γ tuning

This asymmetry (unfixable bias vs. fixable variance) is the project's central thesis — aligned with Metz et al. 2021 ("Gradients Are Not All You Need," uses Lorenz-like chaos as its motivating example) and Suh et al. 2022 ("Do Differentiable Simulators Give Better Policy Gradients?").

5 Critical confound to avoid

RL learns a feedback policy $u(x,t)$; naive adjoint/BPTT gives an open-loop schedule $u(t)$. These aren't comparable — feedback is inherently better for chaotic control regardless of optimization method, because it re-observes state every step and never has to predict past τ_λ. If "RL vs. naive adjoint" is compared directly and RL wins, that only shows feedback beats feedforward, not that RL beats gradients.

The correct 2×2 design: {open-loop, feedback} $\times$ {gradient-based, gradient-free}

	Open-loop (script)	Feedback (thermostat)
Gradient	Naive adjoint/BPTT (strawman)	**MPC** $\leftarrow$ the real baseline
Gradient-free	Random search	**RL** $\leftarrow$ the method of interest

The meaningful comparison is MPC vs. RL, *not* adjoint vs. RL.

6 What to measure

1. **Gradient signal-to-noise ratio vs. horizon**, x-axis in units of τ_λ (not seconds):

$$\mathrm{SNR} = \frac{\|\mathbb{E}[\hat{g}]\|}{\sqrt{\mathrm{tr}\,\mathrm{Cov}(\hat{g})}}$$

for both estimators. Prediction: BPTT SNR falls off a cliff at $T \approx 2$–$3\,\tau_\lambda$; RL degrades gracefully. A clean version of this plot is the core result.

2. **Bias of each estimator** against a shadowing-based ground truth for the ergodic objective.
3. **Whether the crossover point tracks** ρ (i.e. tracks λ_1 itself) rather than wall-clock T — this is what makes it a causal claim rather than a coincidence.
4. Report the RL discount factor γ in units of τ_λ (γ acts as a soft horizon $\approx 1/(1-\gamma)$ steps) and hold it fixed across sweeps — otherwise a "win" for RL may just be an implicit horizon truncation, the same fix available to BPTT.

7 Code / implementation state

7.1 Patrick's repo (BPTT side)

`github.com/pf0010/Chaos-Research` — `sim.py`/`plots.py` structure with shared constants (σ, ρ, β).

- Functions reviewed: `array_data`, `tensor_data`, `create_controller`, `control_force`, `phi`/`calculate_loss`, `position_gradient`, `optimize_gradient`.
- Produces three figures: attractor, Lyapunov-time reference, and a "gradient divergence" separation plot. **No control input is actually defined yet** — all three are uncontrolled diagnostics.
- **Open discrepancy:** the third plot is labeled "gradient divergence" but its y-axis reads as plain state separation, with a slope implying $\lambda \approx 1.4$ (vs. reference 0.906, $\sim$50% high). Either mislabeled, or the fit window includes transient, or δ_0 was set too large (10^{-5}) leaving too few clean decades. Not yet resolved with Patrick.
- **Training-run findings (uncontrolled/degenerate result):**
  - Converges to a degenerate controller: mean $|u| \approx 124$ — essentially overwriting the dynamics with a large constant bias rather than steering it, because the loss has **no control-cost term**, so Adam keeps inflating the bias indefinitely.
  - Rolled out to 10× the training horizon, state diverges to NaN — only "works" on the exact steps it was fit to (no generalization).
  - A hand-tuned one-parameter controller outperforms the learned one.
  - Hardcoded Lyapunov exponent in the codebase (0.9056) disagrees with an independent Euler-discretized measurement (~ 0.958) — meaning every horizon label in the codebase is off by roughly 5%. The RK4-based fit should be treated as more reliable going forward.
  - Central empirical result to date: **gradient direction agreement across different starting points collapses to near zero by ~ 2 Lyapunov times, while loss consistency keeps improving over that same range** — this is the finding motivating the RL comparison.
  - Fixes needed before any BPTT-vs-RL comparison is meaningful: add a control-cost term to the loss (e.g. `loss = phi(x).mean() + alpha*(u**2).mean()`), bound/report actuation magnitude, evaluate on held-out initial conditions and horizons longer than training.

7.2 Your `lorenz.py` (RL side, Step 1 deliverable for Pulkit)

- RK4 fixed-step integrator, $\sigma = 10$, $\rho = 28$, $\beta = 8/3$, float64, burn-in to drop the transient before measuring.
- Four figures: (1) the attractor from a few trajectories, (2) $x(t)$ for a pair separated by $\delta_0 = 10^{-6}$ showing divergence, (3) log-separation with a fitted Lyapunov exponent, (4) an ensemble Lyapunov estimate averaged over ∼100 pairs (single-realization fits are jagged and have no error bar; ergodic averaging over many pairs gives a cleaner exponent).
- Structured to mirror Patrick's repo layout (`sim.py`/`plots.py`, shared constant names) for easy comparison later.

Design-choice justifications

- *Fixed-step RK4 over adaptive solvers:* adaptive stepping makes the two trajectories take different step sequences, contaminating measured separation with integrator disagreement rather than true dynamical divergence.
- *Burn-in:* perturbation growth is only exponential at rate λ once on the attractor and aligned with the leading Lyapunov direction; fitting through the transient inflates the slope.
- *Windowed fit for λ:* below $\sim 10\delta_0$ you're measuring alignment (not yet exponential), above $\sim O(1)$ you're measuring the attractor's diameter (saturated); only the middle region is genuinely exponential.
- *Ensemble estimate:* local expansion rate varies by region of the attractor; λ is a time-average, so a single realization gives a jagged, unreliable curve.
- **Known gap:** the standard rigorous way to compute λ is **Benettin's algorithm** (periodically renormalizing the perturbation vector so it never leaves the linear regime), rather than fitting a single separation curve — flagged as something to know exists even if not yet implemented.

8 RL track specifics

- **Recommended algorithm: SAC** (off-policy, entropy regularization suits the exploration needs of a chaotic/sensitive system, doesn't backprop through the dynamics).
- **Baseline: PPO. Alternative: TD3.**
- Discrete-action methods were ruled out (control is continuous/analog forcing).
- Next planned step (as of last session): sketching a SAC implementation mirroring Patrick's `sim.py`/`plots.py` structure.

9 Reading list

- Metz et al. 2021, *"Gradients Are Not All You Need"* — states the project's core premise; uses Lorenz-like chaos as its motivating example.
- Suh et al. 2022, *"Do Differentiable Simulators Give Better Policy Gradients?"* — rigorous bias/variance treatment of the BPTT-vs-RL asymmetry.